



Bacharelado em Ciência da Computação
Componente: Computação Gráfica → Professor: Claudio Kleina
Aula 3.1 – Mover objetos “dentro” de outros objetos

Respostas das atividades da aula 3:

Questões 5 e 6:

--- 1. Variáveis Globais ---

pos_x = 0.0

pos_y = 0.0

fov = 45.0 # Valor inicial do Field of View

def tratar_teclado(window, key, scancode, action, mods):

 global pos_x, pos_y, fov

 # Velocidade do movimento e do zoom

 passo = 0.1

 passo_fov = 2.0

 if action == glfw.PRESS or action == glfw.REPEAT:

 # Movimentação (Setas) com Limites Simples

 # Nota: Os limites dependem da distância Z e do FOV,

 if key == glfw.KEY_LEFT and pos_x > -2.5:

 pos_x -= passo

 if key == glfw.KEY_RIGHT and pos_x < 2.5:

 pos_x += passo

 if key == glfw.KEY_UP and pos_y < 2.0:

 pos_y += passo

 if key == glfw.KEY_DOWN and pos_y > -2.0:

 pos_y -= passo

 # 2. Controle de FOV (W e S)

 # aqui usamos valores aproximados para um Z de -5.

 if key == glfw.KEY_W:

 fov = max(5.0, fov - passo_fov) # Zoom In (diminui ângulo)

 if key == glfw.KEY_S:

 fov = min(120.0, fov + passo_fov) # Zoom Out (aumenta ângulo)

#Chamar a função depois de criar a janela

 glfw.make_context_current(janela)

 # Registra a função de teclado

glfw.set_key_callback(janela, tratar_teclado)

Código para controlar o Cubo pelo teclado

import glfw

from OpenGL.GL import *

from OpenGL.GLU import *

--- 1. Variáveis Globais ---

```

pos_x = 0.0
pos_y = 0.0
fov = 45.0 # Valor inicial do Field of View

def tratar_teclado(window, key, scancode, action, mods):
    global pos_x, pos_y, fov

    # Velocidade do movimento e do zoom
    passo = 0.1
    passo_fov = 2.0

    if action == glfw.PRESS or action == glfw.REPEAT:
        # Movimentação (Setas) com Limites Simples
        # Nota: Os limites dependem da distância Z e do FOV,
        # aqui usamos valores aproximados para um Z de -5.
        if key == glfw.KEY_LEFT and pos_x > -2.5:
            pos_x -= passo
        if key == glfw.KEY_RIGHT and pos_x < 2.5:
            pos_x += passo
        if key == glfw.KEY_UP and pos_y < 2.0:
            pos_y += passo
        if key == glfw.KEY_DOWN and pos_y > -2.0:
            pos_y -= passo

        # 2. Controle de FOV (W e S)
        if key == glfw.KEY_W:
            fov = max(5.0, fov - passo_fov) # Zoom In (diminui ângulo)
        if key == glfw.KEY_S:
            fov = min(120.0, fov + passo_fov) # Zoom Out (aumenta ângulo)

def desenhar_cubo():
    glBegin(GL_QUADS)
    # Face Frontal (Vermelha)
    glColor3f(1, 0, 0)
    glVertex3f(-1, -1, 1); glVertex3f( 1, -1, 1); glVertex3f( 1, 1, 1); glVertex3f(-1, 1, 1)
    # Face Traseira (Verde)
    glColor3f(0, 1, 0)
    glVertex3f(-1, -1, -1); glVertex3f(-1, 1, -1); glVertex3f( 1, 1, -1); glVertex3f( 1, -1, -1)
    # Face Superior (Azul)
    glColor3f(0, 0, 1)
    glVertex3f(-1, 1, -1); glVertex3f(-1, 1, 1); glVertex3f( 1, 1, 1); glVertex3f( 1, 1, -1)
    # Face Inferior (Amarela)
    glColor3f(1, 1, 0)
    glVertex3f(-1, -1, -1); glVertex3f( 1, -1, -1); glVertex3f( 1, -1, 1); glVertex3f(-1, -1, 1)
    # Face Direita (Magenta)
    glColor3f(1, 0, 1)
    glVertex3f( 1, -1, -1); glVertex3f( 1, 1, -1); glVertex3f( 1, 1, 1); glVertex3f( 1, -1, 1)
    # Face Esquerda (Ciano)
    glColor3f(0, 1, 1)
    glVertex3f(-1, -1, -1); glVertex3f(-1, -1, 1); glVertex3f(-1, 1, 1); glVertex3f(-1, 1, -1)
    glEnd()

```

```
def main():
    if not glfw.init():
        return

    janela = glfw.create_window(800, 600, "Controle de Cubo e FOV", None, None)
    if not janela:
        glfw.terminate()
        return

    glfw.make_context_current(janela)

    # Registra a função de teclado
    glfw.set_key_callback(janela, tratar_teclado)

    glEnable(GL_DEPTH_TEST)

    while not glfw.window_should_close(janela):
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

        # --- Atualização da Projeção (FOV Dinâmico) ---
        glMatrixMode(GL_PROJECTION)
        glLoadIdentity()
        # O FOV agora é controlado pela variável global atualizada pelas teclas W/S
        gluPerspective(fov, 800/600, 0.1, 50.0)

        glMatrixMode(GL_MODELVIEW)
        glLoadIdentity()

        # --- Transformações ---
        # Aplica a posição controlada pelas setas
        glTranslatef(pos_x, pos_y, -5)
        glRotatef(glfw.get_time() * 50, 1, 1, 0)

        desenhar_cubo()

        glfw.swap_buffers(janela)
        glfw.poll_events()

    glfw.terminate()

if __name__ == "__main__":
    main()
```

Representação do sistema solar

Para transformar o nosso cubo em um sistema solar simplificado (Sol e Terra), vamos usar o conceito de **hierarquia de transformações**.

No OpenGL, a ordem em que você chama as funções de translação e rotação é fundamental. Para fazer a Terra girar em torno do Sol, primeiro rotacionamos o sistema de coordenadas e depois transladamos a Terra para longe da origem.

glPushMatrix() e glPopMatrix():

No OpenGL você não move apenas um objeto, você move **todo o universo**.

- **glPushMatrix():** É como se você tirasse uma "foto" das coordenadas atuais do seu GPS e a guardasse no bolso. Você diz ao sistema: *"Salve onde eu estou agora, pois vou fazer várias manobras, mas precisarei saber como voltar exatamente para este ponto."*
- **As Transformações:** Você caminha, gira e se afasta (usando glTranslatef, glRotatef).
- **glPopMatrix():** É como se você pegasse aquela foto no bolso, apertasse um botão de teletransporte e **voltasse instantaneamente** para onde estava quando a foto foi tirada.

Por que isso é vital?

Na construção de um Sistema Solar, por exemplo, se você mover a Terra para longe do Sol e depois quiser desenhar Marte, Marte começaria a ser desenhado a partir da posição da Terra, e não do Sol!

O **Push** permite criar hierarquias: Se eu quero que a Lua gire em torno da Terra, eu faço um Push para a Terra e, **dentro** dele (antes do Pop da Terra), eu coloco as transformações da Lua.

```
def desenhar_cubo(): #Substituir todos "1" por uma variável, exceto no glColor3f
```

```
    glBegin(GL_QUADS)    # Instrução para o OpenGL agrupar cada 4 vértices em um quadrado
```

```
        glColor3f(0, 1, 1) # Cor das faces do cubo
```

```
        # --- Face Frontal ---
```

```
        glVertex3f(-1, -1, 1); glVertex3f( 1, -1, 1); glVertex3f( 1, 1, 1); glVertex3f(-1, 1, 1)
```

```
        # --- Face Traseira ---
```

```
        glVertex3f(-1, -1, -1); glVertex3f(-1, 1, -1); glVertex3f( 1, 1, -1); glVertex3f( 1, -1, -1)
```

```
        # --- Face Superior ---
```

```
        glVertex3f(-1, 1, -1); glVertex3f(-1, 1, 1); glVertex3f( 1, 1, 1); glVertex3f( 1, 1, -1)
```

```
        # --- Face Inferior ---
```

```
        glVertex3f(-1, -1, -1); glVertex3f( 1, -1, -1); glVertex3f( 1, -1, 1); glVertex3f(-1, -1, 1)
```

```
        # --- Face Direita ---
```

```
glVertex3f( 1, -1, -1); glVertex3f( 1, 1, -1); glVertex3f( 1, 1, 1); glVertex3f( 1, -1, 1)
```

```
# --- Face Esquerda ---
```

```
glVertex3f(-1, -1, -1); glVertex3f(-1, -1, 1); glVertex3f(-1, 1, 1); glVertex3f(-1, 1, -1)
```

```
glEnd() # Finaliza o envio de vértices para a placa de vídeo
```

Atividade 1: Represente o Sol Rotacionando e a Terra Rotacionando e Transladando: Você irá precisar de 2 quadrados e usar as funções `glPushMatrix()` e `glPopMatrix()`

Atividade 2: A Terra Gira em uma inclinação vertical de aproximadamente 23.5°. Represente essa inclinação no seu trabalho.

Atividade 3 (Desafio): Adicione a Lua Girando em torno da Terra

```
glTranslatef(pos_x, pos_y, -15) # Afastamos mais para ver a órbita
```

```
# Sequência para criar o Sistema Solar
```

```
# --- SOL ---
```

```
glPushMatrix() # "Congela a cena"
```

```
Rotaciona o Sol
```

```
Desenha o cubo
```

```
glPopMatrix() "Volta para a cena"
```

```
# --- TERRA ---
```

```
glPushMatrix()
```

```
Rotação da Órbita (em torno do Sol)
```

```
Translação (Distância do Sol)
```

```
Rotação da Terra (próprio eixo)
```

```
Desenha o cubo
```

```
glPopMatrix()
```

##Código para a atividade do Sistema Solar

```
import glfw
```

```
from OpenGL.GL import *
```

```
from OpenGL.GLU import *
```

```
# --- Variáveis Globais ---
```

```
pos_x = 0.0
```

```
pos_y = 0.0
```

```
fov = 45.0
```

```
def tratar_teclado(window, key, scancode, action, mods):
```

```
    global pos_x, pos_y, fov
```

```
    passo = 0.1
```

```
    if action == glfw.PRESS or action == glfw.REPEAT:
```

```
        if key == glfw.KEY_LEFT: pos_x -= passo
```

```
        if key == glfw.KEY_RIGHT: pos_x += passo
```

```
        if key == glfw.KEY_UP: pos_y += passo
```

```
if key == glfw.KEY_DOWN: pos_y -= passo
if key == glfw.KEY_W: fov = max(5.0, fov - 2.0)
if key == glfw.KEY_S: fov = min(120.0, fov + 2.0)
```

```
def desenhar_cubo(tamanho, r, g, b):
```

```
    """ Desenha um cubo completo com todas as 6 faces """
```

```
    glColor3f(r, g, b)
```

```
    glBegin(GL_QUADS)
```

```
    # Face Frontal
```

```
    glVertex3f(-tamanho, -tamanho, tamanho); glVertex3f( tamanho, -tamanho, tamanho)
```

```
    glVertex3f( tamanho, tamanho, tamanho); glVertex3f(-tamanho, tamanho, tamanho)
```

```
    # Face Traseira
```

```
    glVertex3f(-tamanho, -tamanho, -tamanho); glVertex3f(-tamanho, tamanho, -tamanho)
```

```
    glVertex3f( tamanho, tamanho, -tamanho); glVertex3f( tamanho, -tamanho, -tamanho)
```

```
    # Face Superior
```

```
    glVertex3f(-tamanho, tamanho, -tamanho); glVertex3f(-tamanho, tamanho, tamanho)
```

```
    glVertex3f( tamanho, tamanho, tamanho); glVertex3f( tamanho, tamanho, -tamanho)
```

```
    # Face Inferior
```

```
    glVertex3f(-tamanho, -tamanho, -tamanho); glVertex3f( tamanho, -tamanho, -tamanho)
```

```
    glVertex3f( tamanho, -tamanho, tamanho); glVertex3f(-tamanho, -tamanho, tamanho)
```

```
    # Face Direita
```

```
    glVertex3f( tamanho, -tamanho, -tamanho); glVertex3f( tamanho, tamanho, -tamanho)
```

```
    glVertex3f( tamanho, tamanho, tamanho); glVertex3f( tamanho, -tamanho, tamanho)
```

```
    # Face Esquerda
```

```
    glVertex3f(-tamanho, -tamanho, -tamanho); glVertex3f(-tamanho, -tamanho, tamanho)
```

```
    glVertex3f(-tamanho, tamanho, tamanho); glVertex3f(-tamanho, tamanho, -tamanho)
```

```
    glEnd()
```

```
def main():
```

```
    glfw.init()
```

```
    janela = glfw.create_window(800, 600, "Cubo 3D - Sol e Terra", None, None)
```

```
    glfw.make_context_current(janela)
```

```
    glfw.set_key_callback(janela, tratar_teclado)
```

```
    # CRÍTICO: Ativa o teste de profundidade para o cubo parecer sólido
```

```
    glEnable(GL_DEPTH_TEST)
```

```
    while not glfw.window_should_close(janela):
```

```
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
```

```
        # Configuração da Projeção
```

```
        glMatrixMode(GL_PROJECTION)
```

```
        glLoadIdentity()
```

```
        gluPerspective(fov, 800/600, 0.1, 100.0)
```

```
        glMatrixMode(GL_MODELVIEW)
```

```
        glLoadIdentity()
```

```
        glTranslatef(pos_x, pos_y, -15) # Afastamos mais para ver a órbita
```

```
# --- SOL ---
```

```
# --- TERRA ---
```

```
glfw.swap_buffers(janela)  
glfw.poll_events()
```

```
glfw.terminate()
```

```
main()
```

Atividade 4 (Desafio): Adicione dois asteroides percorrendo aleatoriamente no espaço. Verifique se houve colisão com algum dos astros, retirando o asteroide da cena e alterando a cor do astro colidido.

Para haver colisão no espaço 3D é necessário que ocorra, simultaneamente:

- $A.minX \leq B.maxX$ e $A.maxX \geq B.minX$
- $A.minY \leq B.maxY$ e $A.maxY \geq B.minY$
- $A.minZ \leq B.maxZ$ e $A.maxZ \geq B.minZ$

Bom estudo!